DEAD PLAYER WALKING

COMPLETE GAME DESIGN DOCUMENT

SECTIONS I–VIII

Genre: Roguelike / Action
Platform: Mobile
Engine: Godot

Studio: Tokens
De Guzman, Asher Nathaniel E.
Macaraeg, Mark Cyrus P.
Zapanta, Zeus Levi C.

Revised consolidated design: 10-floor dungeon, two-weapon combat, permanent upgrades, and stat-based previous-self enemies.

DOCUMENT STATUS AND CONSOLIDATION NOTE

This file consolidates the Dead Player Walking design into one document and updates the mechanics to match the latest direction supplied by the team. The previous-self system no longer records or reproduces player movement, button inputs, attack timing, or playstyle. A previous self is created only from a snapshot of the defeated character's statistics and equipped weapon archetypes, then controlled by standard enemy AI.

Sections V and VI were available as source documents and have been revised to fit the new dungeon structure and economy. The earlier Section IV and Sections VII–VIII content were also aligned to the revised design. Sections I–III are organized here as a complete, consistent opening foundation based on the project concept established in the conversation; compare them with any original class-approved versions before final submission if exact earlier wording is required.

All numerical values identified as provisional are starting balance targets for prototyping, not playtest results. The team must validate them in Godot and update the document after testing.

Current design decision	Rule used throughout this document
Previous-self enemy	Generated from a defeated run's stat snapshot and weapon archetypes only. No behavior recording or playstyle cloning.
Player weapons	The player carries one unique melee weapon and one unique ranged weapon and can switch between them during combat.
Progression	After a run ends, the player can spend earned currency on permanent character-stat upgrades and separate weapon upgrades.
Dungeon structure	A run consists of 10 floors. Floors become larger and contain more rooms as progression advances.
Room combat	Every combat room contains enemies. Entering a room locks its exits until all enemies are defeated.
Healing	No safe or recovery rooms. A limited-use healing pickup can appear in a room and restores a meaningful portion, but never all, of the player's HP.
Boss floors	Floor 5 and Floor 10 are boss floors. The boss is a stronger, higher-HP version of the player's own character.
First boss reveal	The first time the player reaches a boss floor, a short cutscene reveals the self-boss before the fight.

I. GAME OVERVIEW

1.1 Game Title and Basic Information

Item	Description
Title	Dead Player Walking
Genre	Roguelike / Action
Platform	Mobile (Android as the initial target; other mobile platforms may be considered later)
Engine	Godot
Perspective	2D top-down
Players	Single-player
Core setting	A hostile dungeon made of connected combat rooms across 10 floors
Core hook	When a run ends, the defeated character can return as a stat-based enemy in a later run.

1.2 High Concept

Dead Player Walking is a mobile top-down action roguelike in which the player fights through a 10-floor dungeon using two carried weapons: one melee weapon and one ranged weapon. The player can switch between them at any time during combat. Each floor contains multiple enemy-filled rooms that lock when entered and unlock only after all enemies are defeated. Floors grow larger and more demanding as the player descends.

When the player loses, the run's character statistics and weapon archetypes are saved as a snapshot. That snapshot can be used to create a previous-self enemy in a later run. The enemy does not imitate the player's actual movement or combat decisions; it uses a standard AI behavior set while reflecting the defeated character's build. After a run, the player can use earned currency to permanently improve character statistics and upgrade the melee and ranged weapons separately.

Every fifth floor is a boss floor. The boss is the player's own previous self, presented as a deliberate skill check with more HP and greater damage than the normal player character. The first time the player reaches a boss floor, a short cutscene introduces the self-boss and explains the central premise.

1.3 Core Design Pillars

Pillar	Design meaning	How it appears in play
The past returns	A defeated run remains relevant without complex behavior simulation.	A later enemy is generated from the previous character's stats and weapon types.
Two-weapon combat	The player has a clear tactical choice during every encounter.	Switch between the single melee weapon and single ranged weapon carried by the character.
Room-by-room pressure	Each room is a contained combat challenge.	Entering a room locks its exits until every enemy is defeated.
Long-term growth	A failed run still contributes to future attempts.	Currency earned during a run can be spent on permanent stats and weapon upgrades.
A demanding descent	The dungeon becomes more challenging floor by floor.	Floors add rooms, space, enemy groups, and increasingly demanding boss checks.

1.4 Target Player Experience

- Combat should feel responsive and readable on a phone screen.
- The player should regularly decide whether melee or ranged combat is safer or more effective.
- Clearing a room should provide a clear sense of completion and forward momentum.
- Defeat should feel like a setback rather than wasted time because permanent upgrades remain available.
- Meeting a previous self should feel personal because the enemy reflects the player's earlier build, even though it uses simplified AI.
- Boss encounters should test the player's understanding of movement, weapon switching, attack range, and survival.

1.5 Scope and Development Targets

Included in initial scope	Excluded from initial scope
One playable character	Multiple playable character classes
One unique melee weapon	Large weapon catalog or randomized weapon drops
One unique ranged weapon	Multiple ranged weapon types
10 dungeon floors	Endless dungeon or post-floor-10 campaign
Two boss encounters: Floor 5 and Floor 10	Many unique boss families
Stat-based previous-self enemy	Recording movement, inputs, attack sequences, or playstyle

Permanent character and weapon upgrades	Complex skill trees, crafting, or equipment inventory
Room-based enemy encounters and occasional healing pickups	Safe rooms, shops, rest rooms, or healing rooms
Godot 2D mobile prototype	Online multiplayer or network features

II. GAMEPLAY SYSTEMS AND MECHANICS

2.1 Core Player Actions

Action	Description	Gameplay purpose
Move	Move the character in four or eight directions using a virtual joystick.	Positioning, evasion, and navigation.
Melee attack	Attack nearby enemies with the one unique melee weapon.	High-risk close-range damage and crowd control.
Ranged attack	Fire the one unique ranged weapon at enemies from a distance.	Safer damage and engaging enemies before approaching.
Switch weapon	Alternate between the carried melee and ranged weapons.	Adapt to distance, enemy pressure, and room layout.
Dodge / evade	Use movement and/or a dedicated dodge action if retained in the combat prototype.	Avoid attacks and reposition.
Interact	Open the exit or activate a floor transition when the floor is cleared.	Advance through the dungeon.

2.2 Two-Weapon Loadout

The player always carries both weapons. The initial game contains exactly one unique melee weapon and one unique ranged weapon. These are the player's permanent weapon identities for the prototype; the player does not find or equip a different weapon type during a run. Both weapons have independent upgrade levels.

Weapon slot	Initial weapon concept	Strength	Limitation	Upgrade focus
Melee	A single-blade sword (working name: Dungeon Edge)	High damage at close range; useful when enemies approach or become surrounded.	Requires the player to enter danger range.	Damage, attack speed, and optional attack reach.
Ranged	A compact hand-crossbow (working name: Warden Crossbow)	Can damage enemies from a safer distance and help target dangerous enemies.	Slower firing or reload interval; less effective if enemies get too close.	Damage, reload/fire rate, and optional projectile speed.

The weapon names and exact numerical properties are provisional. The final design should keep each weapon's role distinct so that switching has a meaningful tactical purpose.

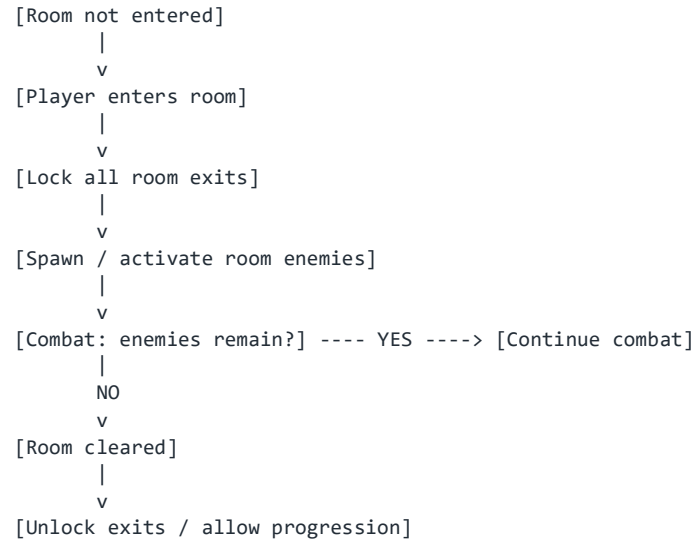
2.3 Weapon Switching Rules

- Both weapons remain in the player's inventory for the entire run.
- Switching is available through a dedicated mobile button or a two-slot weapon toggle.
- Switching should be quick and should not require opening a pause or inventory menu.
- The HUD clearly highlights the currently active weapon.
- Each weapon uses its own damage and upgrade values.
- A previous-self enemy uses the weapon archetype information from its saved snapshot; it does not copy the player's recorded decisions.

2.4 Room Lock and Clear System

Dungeon floors are made of connected rooms. Every room is a combat room and contains at least one enemy group. When the player crosses the room entrance, the room enters a locked combat state: doors or exits close, and the player must defeat all enemies in that room before leaving. When the final enemy is defeated, the room-clear state activates, exits unlock, and the player may continue.

Figure 2.1. Room Encounter State Diagram



Healing pickups do not turn a room into a safe room. They are placed within the normal dungeon flow and may be collected while the player is navigating or fighting, subject to the pickup's placement and room-lock rules.

2.5 Healing Pickup

A one-time-use healing pickup may appear in selected locations across a floor. It restores a meaningful but partial amount of the player's maximum HP. It must never restore the player's HP to full automatically.

Property	Proposed rule
Use	One-time collection; disappears after being collected.
Effect	Restore 30% of maximum HP, rounded to a whole number.
Full-heal prevention	If the player is missing less than the pickup amount, restore only up to maximum HP; do not grant overheal.
Placement	Placed in a room or connecting area as part of the floor layout; no dedicated recovery room.
Availability	Not guaranteed on every floor. The initial prototype can use fixed placement for repeatable testing.
Purpose	Provide a limited recovery opportunity without removing the pressure of room-to-room survival.

2.6 Previous-Self Enemy System

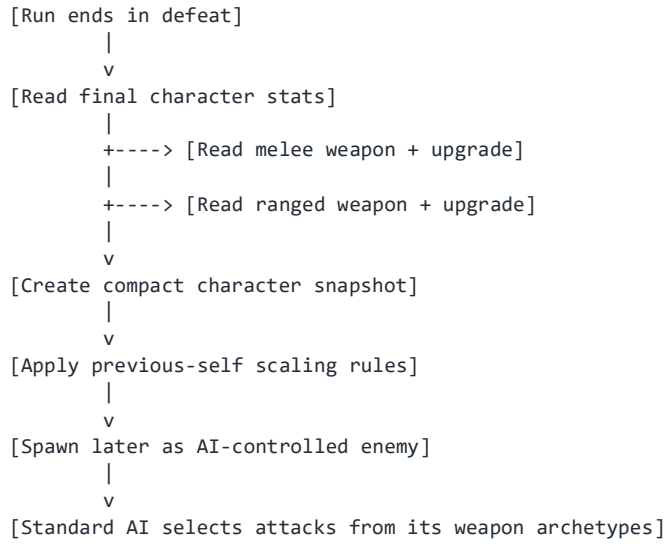
When a run ends in defeat, the game saves a compact character snapshot. The snapshot contains only the character's relevant numerical stats and the identities/upgrade levels of the two weapon archetypes. It does not contain a movement recording, input history, attack timing, dodge history, room route, or playstyle profile.

Saved snapshot field	Purpose
Maximum HP / relevant defensive stat	Determines the previous-self enemy's durability.
Character damage modifier or base attack stat	Contributes to the enemy's damage scaling.

Movement speed (if used by the player stat system)	Sets a bounded movement-speed value for the enemy.
Melee weapon identity and upgrade level	Determines which melee attack the enemy AI can use and its scaled damage.
Ranged weapon identity and upgrade level	Determines which ranged attack the enemy AI can use and its scaled damage.
Snapshot version / run identifier	Allows the game to validate and load the correct data.

The previous-self enemy uses a small set of standard AI states such as approach, keep distance, attack, recover, and reposition. These are designed enemy behaviors, not reconstructed player behavior.

Figure 2.2. Stat-Based Previous-Self Creation



For ordinary previous-self enemies, the starting balance target is to make them slightly weaker than the current player so that they are threatening but fair. A provisional target is 85% of the snapshot's maximum HP and 85–90% of its weapon damage, with movement speed capped to a value comparable to ordinary enemies. These values must be tested against actual upgrade progression.

Boss versions are intentionally different. On Floor 5 and Floor 10, the self-boss has more HP and somewhat higher damage than the current player, creating a skill check. The boss uses the same stat-and-weapon snapshot concept but applies boss-specific scaling and a more deliberate AI state sequence.

2.7 Run End and Between-Run Progression

When the player dies, the current run ends. The player is shown a summary of floors reached, enemies defeated, currency earned, and upgrades available. The player can spend accumulated currency on permanent character-stat upgrades and on the independent upgrade tracks for the melee and ranged weapons. After spending or choosing to continue, the player starts a new run.

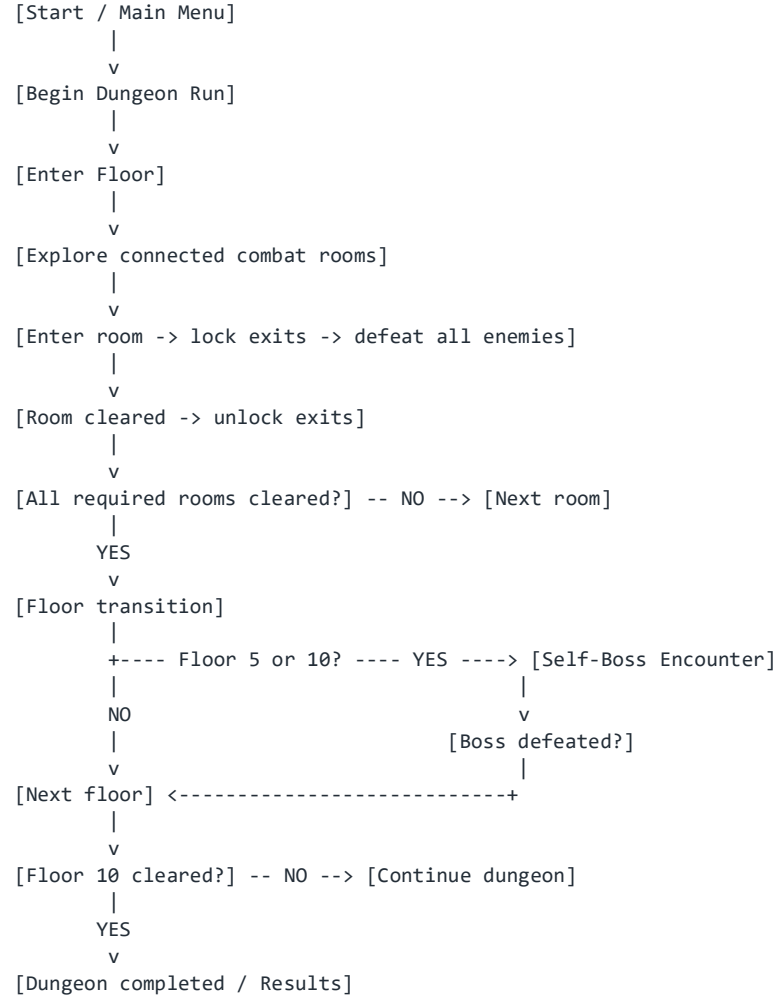
A completed floor grants a reward. If the player dies before completing the full 10-floor dungeon, the player keeps the currency earned during that run according to the proposed persistence rule below. The death screen may also grant a small fixed consolation reward so that early deaths still contribute to progression.

Outcome	Reward treatment
Clear an ordinary floor	Award a fixed floor-clear currency reward.
Defeat a floor boss	Award a larger boss-clear reward.
Die during a floor	Keep currency already earned during the run and receive a small death reward.
Complete Floor 10	Award the final completion reward and show the run completion summary.

III. GAME FLOW, PLAYER PROGRESSION, AND GAME CONTENT

3.1 Main Game Flow

Figure 3.1. Main Game Flow



3.2 Ten-Floor Dungeon Structure

The initial release scope contains 10 floors. Each floor is larger than the early floors through an increase in room count, room dimensions, enemy combinations, or a combination of these factors. The player should feel that the dungeon is descending into a more dangerous area.

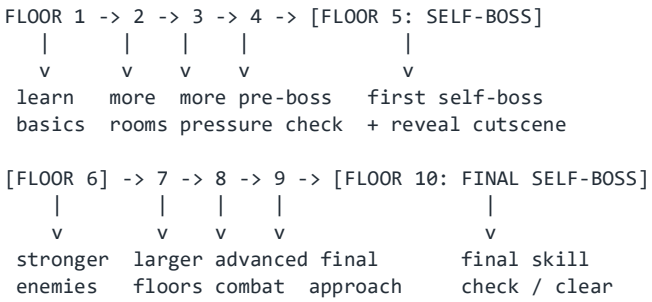
Floor	Floor role	Proposed room count	Design focus
1	Introduction	4 rooms	Teach movement, room locks, basic enemies, melee/ranged switching, and the first healing pickup.
2	Combat expansion	5 rooms	Introduce more enemy combinations and encourage switching weapons.
3	Pressure increase	6 rooms	Larger rooms and enemies that pressure both close and long range.
4	Pre-boss challenge	7 rooms	Combine familiar enemy types and test resource

			management before the first boss.
5	Boss floor 1	Boss arena plus approach	First self-boss encounter; first-time cutscene reveals the player will face themselves.
6	Dungeon escalation	7 rooms	Resume regular combat with stronger enemy combinations.
7	Larger floor	8 rooms	Increase room size and encounter complexity.
8	Advanced combat	9 rooms	Require more deliberate weapon switching and healing-pickup management.
9	Final approach	10 rooms	Highest ordinary-floor pressure; prepare the player for the final self-boss.
10	Final boss floor	Boss arena plus approach	Final self-boss skill check and completion of the initial 10-floor scope.

Room counts are provisional level-design targets. The team may revise them after measuring average mobile session length and Godot production effort. The key requirement is that later floors feel larger through more rooms and/or larger room footprints, not merely by increasing enemy HP.

3.3 Floor Progression Diagram

Figure 3.2. Floor and Boss Progression



3.4 Boss Design

Boss floors occur every five floors: Floor 5 and Floor 10. Both bosses are versions of the player's own character. The first time the player reaches Floor 5, a short cutscene introduces the self-boss. The scene should establish that the dungeon can produce an enemy version of the player's own build. The cutscene is shown only on the first-ever arrival at a boss floor, not on every retry, unless the team decides a shorter replayable version is needed.

Boss property	Floor 5 boss	Floor 10 boss
Identity	The player's self-boss	The player's self-boss / final version
HP	Greater than the player's current maximum HP	Greater than Floor 5 boss and clearly above the current player's HP
Damage	Somewhat higher than the player's normal damage	Higher than Floor 5 boss, tuned as the final skill check
Weapons	Uses the melee and ranged weapon archetypes	Uses both weapon archetypes with more demanding attack sequencing
AI	Readable standard boss states: approach, melee attack, create distance, ranged attack, recovery	Same readable states with tighter windows and stronger pressure

Purpose	Teach the player to fight a stronger version of their own build	Test mastery of movement, switching, spacing, and upgrades

The self-boss is intentionally stronger than the player. A provisional tuning target is 125–140% of the player's maximum HP and 110–120% of the player's weapon damage. These are initial design targets only. The boss must telegraph attacks clearly and provide response windows so the fight is difficult but learnable rather than arbitrary.

3.5 Player Progression

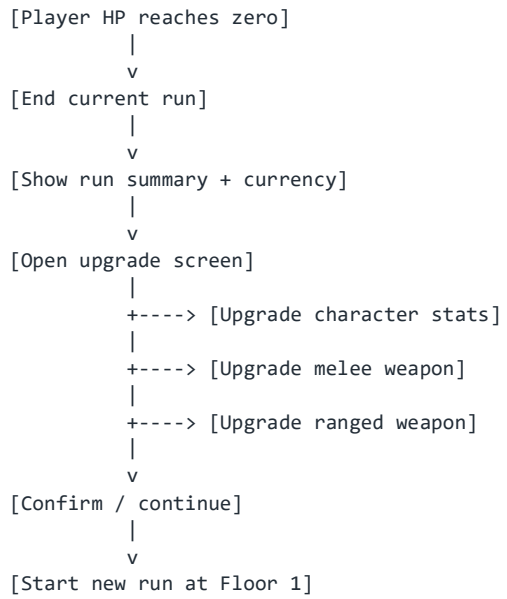
Progression is persistent across runs. The player can improve character statistics and upgrade each weapon independently. Character upgrades affect the player regardless of which weapon is active; weapon upgrades affect only the corresponding weapon.

Upgrade category	Example upgrade	Effect
Character: HP	Maximum HP +10 per level	Improves survivability in rooms and boss encounters.
Character: damage / power	Global attack modifier + small fixed amount	Improves the player's overall offensive output if a global stat is retained.
Character: movement	Movement speed + small fixed amount	Improves positioning and evasion; must have a cap.
Melee weapon	Melee damage + per level	Improves the unique melee weapon only.
Melee weapon	Melee attack speed + per level	Reduces the time between melee attacks, within a safe cap.
Ranged weapon	Ranged damage + per level	Improves the unique ranged weapon only.
Ranged weapon	Reload / fire interval + per level	Improves ranged firing cadence, within a safe cap.

The first prototype should limit the number of upgrade types to avoid an overly complex economy. At minimum, implement HP, melee damage, and ranged damage upgrades. Add attack-speed and movement upgrades only if the team has time to tune them.

3.6 Game Over and Restart Flow

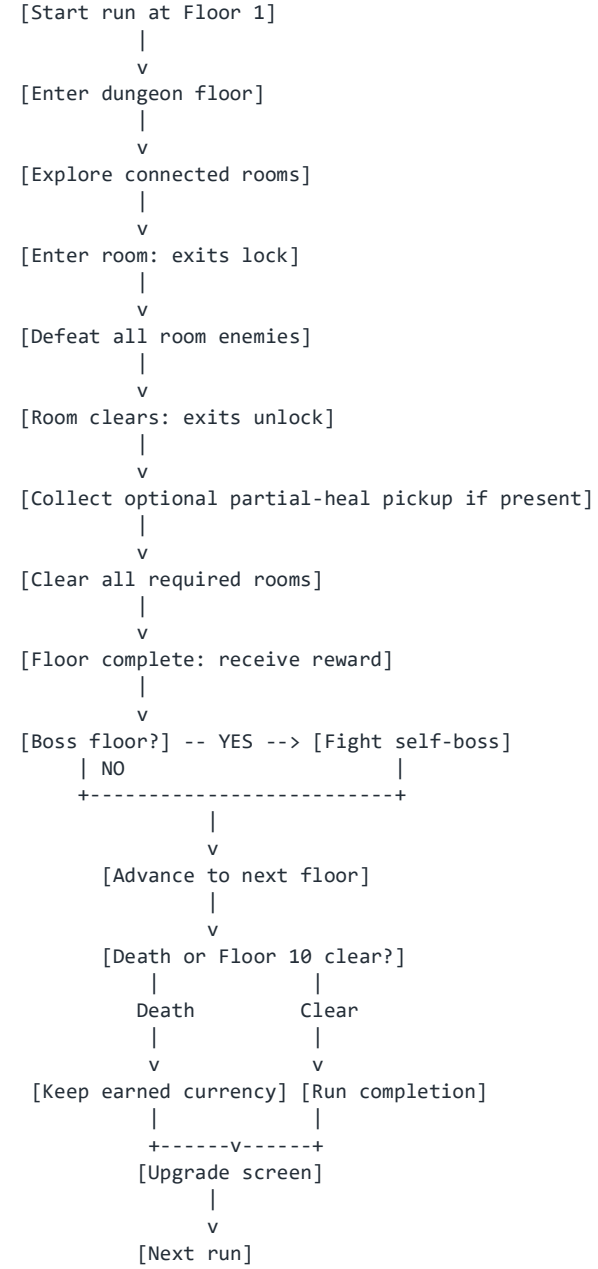
Figure 3.3. Death and Upgrade Flow



IV. CORE GAMEPLAY LOOP

4.1 Core Loop Diagram and Walkthrough

Figure 4.1. Core Gameplay Loop



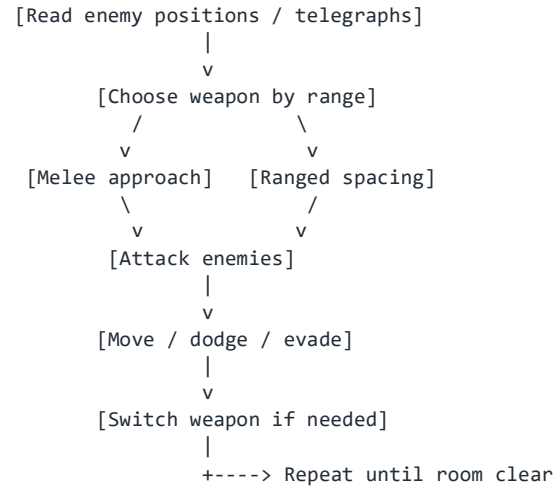
Step	Description
Start run	The player starts at Floor 1 with permanent character stats and both upgraded weapon slots.
Enter floor and rooms	The player navigates a connected dungeon layout. Every combat room contains enemies.
Room lock	On entry, exits lock and enemies activate. The player must defeat all enemies to unlock the room.
Combat and weapon switching	The player moves, attacks, dodges/repositions, and switches between melee and ranged weapons.

Healing pickup	A partial one-time-use heal may be found in a room; it is not a guaranteed full recovery.
Floor completion	After all required rooms are cleared, the player receives the floor reward and proceeds.
Boss encounter	On Floors 5 and 10, the player fights a stronger self-boss. The first boss-floor arrival includes the reveal cutscene.
Run end and upgrades	After death or completion, the player reviews the run and spends currency on permanent stats and weapon upgrades.

4.2 Micro Gameplay Loop

The micro loop describes moment-to-moment combat decisions inside a locked room.

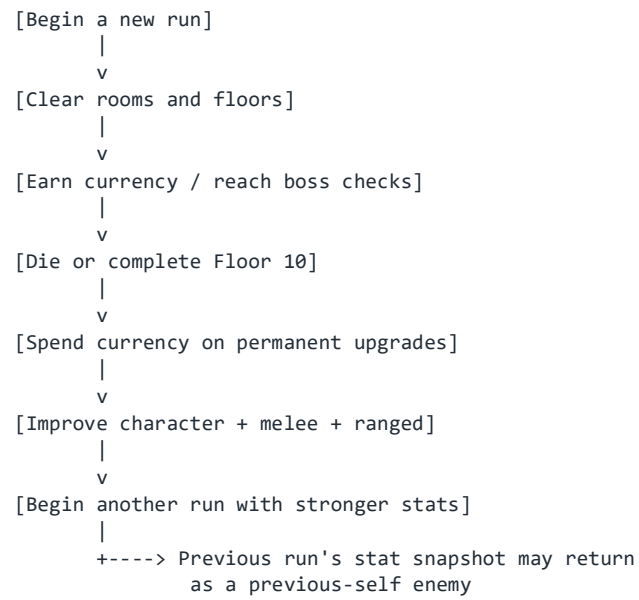
Figure 4.2. Micro Combat Loop



4.3 Macro Gameplay Loop

The macro loop describes long-term progression across floors and repeated runs.

Figure 4.3. Macro Progression Loop



4.4 In-Game and Out-of-Game Actions

In-game actions	Out-of-game actions
Move through dungeon rooms	Review run results
Fight room enemies	Spend currency on permanent HP/stat upgrades
Switch melee and ranged weapons	Upgrade melee weapon independently
Dodge, reposition, and avoid attacks	Upgrade ranged weapon independently
Collect partial healing pickup	Review previous-self snapshot availability
Fight Floor 5 and Floor 10 self-bosses	Start the next run

The micro loop supports the macro loop: effective movement, weapon selection, and room clearing allow the player to reach later floors. The macro loop supports repeated attempts by making currency and permanent upgrades meaningful even after defeat.

V. LEVEL DESIGN

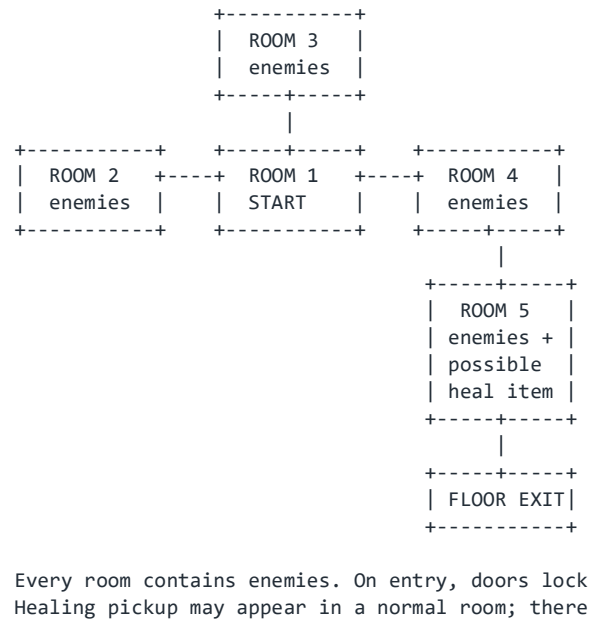
5.1 Level Layout Pattern

Chosen pattern: connected room-based dungeon layout with a mostly linear floor progression. Each floor consists of several rooms connected by doors or passages. The player can navigate the floor's room network, but each combat room becomes a contained arena when entered. Its exits remain locked until every enemy is defeated.

This structure keeps the tactical clarity of a room-based action game while allowing later floors to grow in size and room count. It also supports a controlled difficulty curve and is suitable for a mobile screen.

5.2 Standard Floor Room Flow

Figure 5.1. Example Floor Layout (conceptual)



Every room contains enemies. On entry, doors lock until all room enemies are defeated.
Healing pickup may appear in a normal room; there are no safe/recovery rooms.

The diagram is illustrative rather than a fixed final map. Room connections and dimensions can change between floors, but every floor must maintain the room-lock rule and must not include empty recovery rooms.

5.3 Floor Size and Difficulty Progression

Floor	Room count target	Size / encounter	Difficulty target (1–5)

		progression	
1	4	Small rooms; low enemy count; teach movement, attacks, switching, and locked rooms.	1.0
2	5	Slightly larger footprint; add enemy combinations.	1.5
3	6	More room connections and enemies that challenge both weapon ranges.	2.5
4	7	Pre-boss pressure; tougher groups and more careful health management.	3.0
5	Boss arena	Self-boss with more HP and damage than player; first-time reveal cutscene.	4.5
6	7	Return to regular rooms with stronger enemy combinations.	3.0
7	8	Larger floor footprint and increased enemy count.	3.5
8	9	More demanding room encounters and reduced healing certainty.	4.0
9	10	Final approach with high pressure and limited room for mistakes.	4.5
10	Boss arena	Final self-boss; strongest skill check in initial scope.	5.0

Difficulty values are targets for testing, not measured results. Boss floors are intentionally harder spikes. After each boss, the next ordinary floor should still be challenging but should not feel like an immediate second boss.

5.4 Teaching Mechanics Through Level Sequencing

Mechanic	First introduction	Teaching method	Expected player learning
Movement and room navigation	Floor 1, first room	Wide space and clearly visible exits.	Understand movement and room boundaries.
Room lock / clear rule	Floor 1, first enemy room	Doors visibly close on entry and open after the last enemy is defeated.	Understand that each entered room must be cleared.
Melee attack	Floor 1, early room	Small number of low-threat enemies approach the player.	Use melee when enemies are close.
Ranged attack	Floor 1, early room	Place an enemy at a distance that is safer to engage with range.	Use ranged attacks to control distance.
Weapon switching	Floor 1, mixed enemy room	Place enemies at different distances in the same encounter.	Switch weapons based on tactical situation.
Partial healing pickup	Floor 1 or 2	A visible one-time-use pickup restores a portion of missing HP.	Understand healing is limited and should be collected strategically.
Previous-self enemy	After first defeat / suitable later encounter	Show an enemy using the player's stat and weapon archetype snapshot.	Understand that a defeated build can return as an enemy.

Self-boss	Floor 5	First-time cutscene, then a readable boss fight.	Understand the boss as a stronger version of the player's own build.

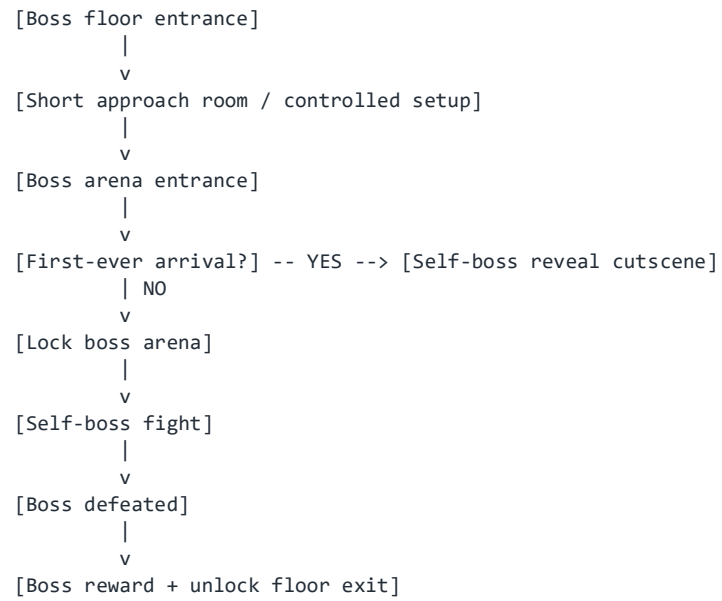
5.5 Healing Pickup Placement

Healing pickups are placed in ordinary dungeon rooms, not in dedicated recovery rooms. The pickup is a one-time-use object and restores a meaningful portion of the player's missing HP without fully healing the character. Placement should be visible enough to reward awareness but should not make the room safe or trivial.

- Do not guarantee a heal in every room or every floor.
- Do not place the pickup directly on the room entrance where it can be collected without engaging with the room.
- Do not place it in a way that forces the player to take unavoidable damage to collect it.
- For early testing, use fixed placements; randomize only after the floor layouts and difficulty are stable.

5.6 Boss Floor Layout

Figure 5.2. Boss Floor Flow



The approach area is not a recovery room. It is a short transition space leading to the boss. The boss arena locks once the fight begins and opens after the boss is defeated.

5.7 Pacing Risks and Mitigation

Risk	Why it may happen	Mitigation
Combat fatigue	Every room contains enemies and there are no safe rooms.	Vary room dimensions, enemy combinations, and combat objectives; use short transitions between encounters.
Overwhelming room locks	A player may be trapped with too many enemies.	Introduce enemy counts gradually; provide clear exits and readable telegraphs; cap simultaneous pressure.
Healing scarcity	The player may enter a boss floor with very low HP.	Tune heal placement and enemy damage; keep heals partial but useful; avoid unavoidable damage.
Floor length	Later floors contain more rooms and may create long mobile sessions.	Keep individual rooms compact and allow pause/resume between rooms if technically feasible.

Boss difficulty spike	Self-boss has more HP and damage than the player.	Use telegraphed attacks, recovery windows, and clear hit feedback; tune using repeated playtests.

Section V revision note: This updated level design replaces the earlier proposal for a recovery room and a single linear tutorial level. All floors now use enemy-filled rooms, a room-lock clear condition, and a 10-floor dungeon progression.

VI. SYSTEMS, ECONOMY, AND BALANCE

6.1 Resources, Sources, and Sinks

The economy is intentionally small. The player earns one primary upgrade currency during a run and spends it between runs. Character upgrades and weapon upgrades use the same currency in the initial prototype, keeping the user interface and balancing manageable.

Resource	How it is gained	How it is spent / used	Persistence
Upgrade currency	Floor-clear rewards, boss rewards, and a small death reward.	Permanent character upgrades; melee weapon upgrades; ranged weapon upgrades.	Persists between runs.
Player HP	Starts at the current maximum HP; may be restored by partial healing pickups.	Lost through enemy attacks and hazards.	Current HP resets for a new run; maximum HP upgrades persist.
Previous-self snapshot	Created from final character stats and weapon archetypes when a run ends in defeat.	Used as input data to create a future AI enemy; not spent as currency.	May persist for a bounded number of previous-self encounters.
Melee weapon level	Increased by spending upgrade currency.	Improves melee weapon performance.	Persists between runs.
Ranged weapon level	Increased by spending upgrade currency.	Improves ranged weapon performance.	Persists between runs.

No resource is created by recording behavior. The previous-self snapshot is a small data record of numerical stats and weapon archetypes only.

6.2 Proposed Reward and Upgrade Values

Event / upgrade	Provisional value	Purpose
Clear ordinary floor	+30 currency	Reward steady progress.
Clear Floor 5 boss	+60 currency	Provide a larger reward for the first boss milestone.
Clear Floor 10 boss	+100 currency	Reward completion of the initial dungeon.
Die before clearing a floor	+10 consolation currency, in addition to currency already earned	Ensure a failed run still contributes.
HP upgrade	20 currency: +10 maximum HP	Simple, visible survivability increase.
Melee damage upgrade	20 currency: +2 melee damage	Strengthen the close-range weapon.
Ranged damage upgrade	20 currency: +2 ranged damage	Strengthen the ranged weapon.
Weapon cadence upgrade	25 currency: small attack-speed / reload improvement	Optional secondary upgrade if scope permits.

These values are starting points. The team should not claim that the economy is balanced until paper simulation and actual playtesting have been completed. If a player can afford every upgrade too quickly, increase costs or reduce rewards; if upgrades arrive too slowly, adjust rewards or costs.

6.3 Progression Curve

The player begins with a modest stat baseline and becomes stronger through permanent upgrades. The dungeon challenge rises through more rooms, larger spaces, enemy combinations, and boss encounters. The goal is not to increase enemy stats at the same rate as player upgrades; difficulty should primarily increase through encounter composition and room structure.

Progress stage	Player progression	Dungeon challenge
New save	Base HP and base melee/ranged damage; both weapons available.	Floor 1 teaches controls and room clearing.
After first failed run	Small consolation currency can buy or contribute toward a basic upgrade.	Previous-self snapshot may be eligible for later use.
Early progression	Player can begin investing in HP and one or both weapon upgrades.	Floors 2–4 introduce more rooms and mixed enemy ranges.
First boss milestone	Player should have had opportunities to improve stats and both weapons.	Floor 5 self-boss tests switching, positioning, and survival.
Mid dungeon	Upgrades make the player more capable, but do not remove room threats.	Floors 6–9 add room count, room size, and enemy combinations.
Final milestone	Player enters Floor 10 with accumulated permanent progression.	Floor 10 self-boss is the final skill check.

6.4 Symmetry, Asymmetry, and Fairness

Dead Player Walking is an asymmetric single-player game. The human controls the current character directly, while ordinary enemies, previous selves, and bosses are controlled by AI. The previous-self enemy reflects the player's earlier stats and weapon archetypes, but does not use the same input method or reproduce the player's past actions.

Fairness rule	Implementation
Readable enemy behavior	Enemies use visible, learnable AI states and clear attack telegraphs.
Ordinary previous-self scaling	Slightly weaker than the current player as a starting target; avoid copying upgrades in a way that creates an impossible enemy.
Boss scaling	Boss has more HP and somewhat higher damage than the player, but attacks must have response windows.
Room-lock clarity	Doors visibly close on entry and visibly unlock after the last enemy is defeated.
Healing transparency	Pickup clearly communicates that it restores a partial amount, not full HP.
Upgrade transparency	Show current value, cost, and resulting value before purchase.
Weapon distinction	Melee and ranged weapons have clear strengths and limitations so switching is meaningful.
Bounded enemy pressure	Avoid spawning too many simultaneous attackers in a small room.

6.5 Randomness and Probability

The first playable prototype should prioritize deterministic layouts and rewards. This allows the team to reproduce issues and test room difficulty, currency balance, and boss fairness. Random room generation and random enemy placement are not required for the initial 10-floor scope.

System	Initial rule	Reason
Floor layout	Fixed authored layouts for the prototype.	Makes room flow and difficulty repeatable.

Room enemy placement	Fixed placement or fixed enemy group per room.	Allows controlled testing of room locks and combat balance.
Floor rewards	Fixed reward values.	Makes upgrade pacing predictable.
Healing pickup	Fixed placements during early testing; optional limited variation later.	Prevents random healing from hiding balance problems.
Previous-self snapshot selection	Use a clearly defined eligible defeated-run snapshot; do not choose randomly in the first prototype.	Makes enemy strength predictable.
Boss stats	Deterministic scaling from the relevant player snapshot and boss-floor multiplier.	Ensures repeatable boss testing.

6.6 Playtest and Revision Record

Complete the following fields after actual paper simulation and peer playtesting. Do not invent results.

Item	Record actual result
Simulation setup	[Number of simulated runs, floors reached, currency earned, and upgrades purchased.]
Issue observed	[Specific issue, such as HP upgrades dominating weapon upgrades or Floor 5 boss being too difficult.]
Peer feedback	[Write the actual feedback received from the group that tested the system.]
Revision made	[State the actual numerical or rule change made after testing.]
Reason	[Explain how the change improves progression, clarity, or fairness.]

6.7 Reflection Questions

1. Which upgrade was most valuable during testing, and did it make the other upgrade choices less attractive?
2. Could a player who loses early still make meaningful progress through the currency reward?
3. Was the Floor 5 self-boss difficult but understandable, or did players experience unavoidable damage?
4. Did the room-lock rule create tension without making rooms feel unfairly crowded?
5. Did the partial healing pickup help survival without removing the need to manage HP?

References: University of Baguio, School of Information Technology. (n.d.). Systems and Balance: Drafting GDD Section VI. GAMDES1 laboratory instructional material.

VII. NARRATIVE DESIGN

7.1 Premise and Setting

The player is trapped in a hostile dungeon that descends through 10 floors. Each floor is made of rooms filled with enemies. The dungeon appears to remember those who have failed within it: when a character dies, the character's build can be reconstructed as an enemy. The dungeon does not replay the person's exact actions; it creates a hostile counterpart using the defeated character's stats and weapon types.

The central narrative question is simple: if the player keeps improving, can they eventually defeat the strongest version of themselves? The Floor 5 boss is the first direct reveal of this idea. The first time the player reaches Floor 5, a short cutscene introduces the self-boss. Floor 10 provides the final confrontation.

7.2 Narrative Delivery Method

The primary method is environmental storytelling, following the “show, don't tell” approach discussed in Lesson 10. The dungeon communicates its history through damaged rooms, abandoned equipment, traces of previous battles, and the appearance of previous-self enemies. The game should not depend on long explanatory cutscenes.

Narrative element	What it communicates	Placement
Damaged walls and weapon marks	Many battles have taken place in the dungeon.	Ordinary combat rooms.
Abandoned supplies	Other attempts have ended here.	Room edges and connecting corridors.
A previous-self enemy	The dungeon can reconstruct a defeated character's build.	Selected rooms after the mechanic is introduced.
Floor 5 cutscene	The boss is a version of the player; the central premise becomes explicit.	First-ever arrival at Floor 5.
Increasingly imposing architecture	The dungeon becomes more dangerous as the player descends.	Floors 6–10.
Floor 10 boss arena	The player has reached the final test against a stronger self.	Final floor.

7.3 Floor 5 First-Reach Cutscene

The first time the player reaches Floor 5, the game briefly pauses before the boss fight. The player enters the arena and sees a figure carrying the same melee and ranged weapon types. The figure's appearance and equipment identify it as a version of the player. The camera frames both characters, the boss assumes a combat stance, and the encounter begins.

The cutscene should be short and focused. It introduces the self-boss without requiring dialogue-heavy exposition. After the first viewing, the game can skip the full sequence on later attempts or show a shortened version.

7.4 Branching Narrative and Player Agency

The initial scope uses a mostly linear narrative. The player's meaningful choices are primarily gameplay choices: which weapon to use, when to switch, whether to collect a healing pickup, and which permanent upgrades to purchase. These decisions change how the player experiences the dungeon without requiring a branching dialogue system.

A larger branching narrative is not recommended for the initial prototype because the 10-floor dungeon, two weapons, enemy AI, upgrades, and boss encounters already provide substantial implementation work. If a small narrative choice is added later, it should have a visible consequence and reconverge with the main dungeon progression.

7.5 Narrative Consistency

- The previous-self enemy is a stat-and-weapon reconstruction, not a recording of player behavior.
- The two weapon identities remain consistent across the narrative and gameplay systems.
- The dungeon's 10-floor structure is the main narrative journey.
- Floor 5 is the first self-boss reveal; Floor 10 is the final self-boss encounter.
- The player grows through persistent upgrades while the dungeon continues to present stronger challenges.

VIII. TECHNICAL DESIGN

8.1 2D / 3D Decision

Chosen approach: 2D top-down. This is the most practical option for the mobile action dungeon and the team's initial Godot scope. It keeps room boundaries, enemy positions, projectiles, melee range, pickups, and exits readable on a small screen. It also reduces the complexity of camera collision, depth perception, occlusion, and 3D character animation.

Visual depth may be added through shadows, lighting, sprite layering, and environmental effects, but the game logic and collision system remain 2D.

8.2 Camera System

- Use a top-down Camera2D that follows the player.
- Use smooth follow with a small dead zone to prevent constant camera jitter.

- Clamp the camera to the boundaries of the current floor or room.
- Avoid camera rotation during combat so screen directions remain predictable.
- Keep the player, nearby enemies, room doors, projectiles, and healing pickups visible.
- Use clear visual contrast to distinguish ordinary enemies, previous selves, and bosses.

This follows Lesson 9's emphasis on treating camera, character, and control as one coordinated system.

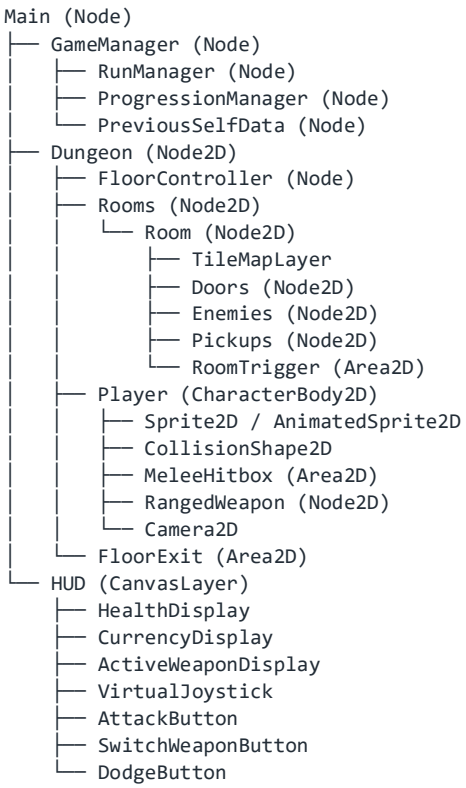
8.3 Mobile Control Scheme and Input Mapping

Mobile input	Action	Implementation notes
Virtual joystick (lower-left)	Move the character.	Screen-oriented movement; consistent response in every room.
Attack button (lower-right)	Attack with the active weapon.	Button label or icon changes with active weapon if useful.
Weapon switch button	Toggle between melee and ranged weapons.	Always available during combat; clear active-weapon indicator.
Dodge button (optional if included in final combat prototype)	Perform a short evade.	Must have clear feedback and a tuned cooldown if used.
Interact / exit prompt	Use the floor exit after required rooms are cleared.	Contextual prompt; should not cover combat space.

The lower-left joystick and lower-right action buttons should be large enough for touch input and spaced to reduce accidental presses. The center of the screen should remain as unobstructed as possible. Simultaneous movement and attack input should be supported where technically practical.

8.4 Godot Scene Tree Plan

Figure 8.1. Proposed Godot Scene Tree



8.5 Main Scene Responsibilities

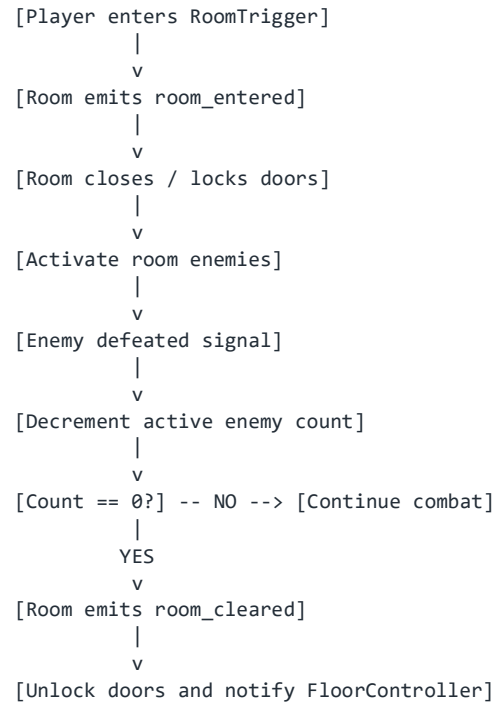
Node / scene	Responsibility
GameManager	Coordinates high-level game state and scene transitions.
RunManager	Tracks current floor, run outcome, currency earned, death, and completion.
ProgressionManager	Stores permanent character stats and weapon upgrade levels.
PreviousSelfData	Stores and validates the defeated character's stat and weapon snapshot only.
FloorController	Loads room layouts, tracks floor completion, and handles floor transitions.
Room	Controls room entry, exit locks, enemy activation, and room-clear state.
Player	Handles movement, health, weapon switching, attacks, and damage.
Melee weapon node	Implements the unique melee weapon's hitbox, damage, and attack timing.
Ranged weapon node	Implements the unique ranged weapon's projectile and firing timing.
Enemy / PreviousSelf	Uses AI states to move and attack; previous-self stats come from the snapshot.
Boss	Uses boss-specific health/damage scaling and a readable boss AI state machine.
HUD	Displays health, currency, active weapon, and mobile controls.

8.6 Signals and Event Wiring

Signal / event	Emitted by	Receiver	Response
room_entered(room_id)	RoomTrigger / FloorController	Room	Lock exits and activate enemies.
enemy_defeated(enemy_id)	Enemy	Room	Reduce active enemy count; check whether the room is cleared.
room_cleared(room_id)	Room	FloorController	Unlock room exits and update floor progress.
player_health_changed(current, max)	Player	HUD	Update health display.
player_died	Player	RunManager	End run, save stat/weapon snapshot, calculate death reward, and open results.
weapon_switched(weapon_id)	Player	HUD	Update active weapon indicator.
floor_cleared(floor_number)	FloorController	RunManager	Award floor currency and load next floor or boss transition.
boss_defeated(floor_number)	Boss	RunManager / FloorController	Award boss reward and unlock the floor exit.
pickup_collected(pickup_type, value)	Pickup	Player / HUD	Apply partial heal or other pickup effect and update display.
upgrade_purchased(upgrade_id)	ProgressionManager	HUD / Upgrade Screen	Refresh currency and displayed stats.

8.7 Room Lock Event Flow

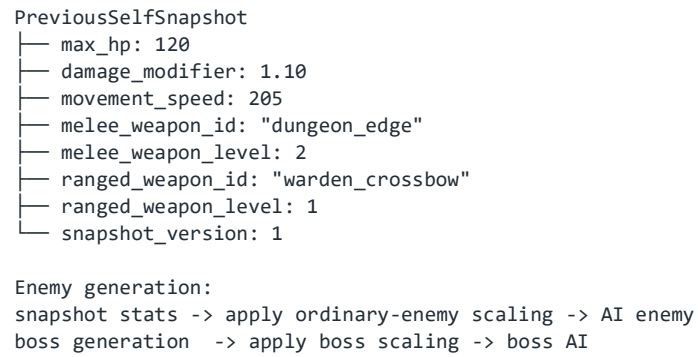
Figure 8.2. Room Signal Flow



8.8 Previous-Self Data Structure and AI

The previous-self data structure should be deliberately small. It stores only the defeated character's relevant stats and the two weapon archetypes with their upgrade levels. No movement path, input events, attack sequence, dodge pattern, room route, or behavioral recording is stored.

Figure 8.3. Example Previous-Self Data (illustrative)



The example values are illustrative only. Actual data fields should match the implemented player-stat system. The AI can use a small finite state machine:

AI state	Behavior
Approach	Move toward the player when outside the preferred attack range.
Melee attack	Use the melee weapon when the player is close enough.
Maintain distance	Move away or reposition when the player is too close for ranged use.

Ranged attack	Use the ranged weapon when line of sight and distance conditions are satisfied.
Recover	Pause briefly after an attack to create a readable response window.
Boss phase / pressure	For bosses, adjust attack frequency or state transitions while preserving visible telegraphs.

8.9 Development and Testing Order

6. Create the top-down player and verify mobile movement.
7. Implement the melee weapon and confirm damage, range, and attack timing.
8. Implement the ranged weapon and confirm projectile behavior and firing interval.
9. Add the weapon-switch control and active-weapon HUD indicator.
10. Build one room with doors that lock on entry and unlock after all enemies are defeated.
11. Add room-to-room floor progression and a floor-clear reward.
12. Add partial healing pickups and verify they never restore more than maximum HP or automatically full-heal.
13. Implement permanent character-stat upgrades and independent weapon upgrades.
14. Implement the stat-and-weapon snapshot and spawn one ordinary previous-self enemy using standard AI.
15. Build the Floor 5 self-boss and first-reach cutscene.
16. Expand the dungeon to 10 floors, increasing room count/size and tuning enemy groups.
17. Build the Floor 10 boss and test the complete 10-floor run.

8.10 Technical Scope Note

This scene tree, signal list, and data flow are proposed implementation plans, not claims that the features have already been built. The team should verify each system in the Godot project and update node names, signal names, and data fields to match the actual implementation.

REFERENCES

Csikszentmihalyi, M. (1990). Flow: The psychology of optimal experience. Harper & Row.

University of Baguio, School of Information Technology. (n.d.). Level Design: Drafting GDD Section V. GAMDES1 laboratory instructional material.

University of Baguio, School of Information Technology. (n.d.). Systems and Balance: Drafting GDD Section VI. GAMDES1 laboratory instructional material.

University of Baguio, School of Information Technology. (n.d.). 2D versus 3D Design Considerations, Camera Systems, and Control Schemes. GAMDES1 Lesson 9.

University of Baguio, School of Information Technology. (n.d.). Narrative Design for Games: Environmental Storytelling and Branching Narrative. GAMDES1 Lesson 10.

Studio Tokens. (n.d.). Dead Player Walking project design materials and prior GDD sections.

FINAL CONSISTENCY CHECKLIST

- Game is designed as a mobile, 2D top-down action roguelike in Godot.
- The dungeon has 10 floors; Floor 5 and Floor 10 are boss floors.
- Every combat room contains enemies and locks its exits until all enemies are defeated.
- Floors become larger and contain more rooms as the player descends.
- There are no dedicated safe, rest, or recovery rooms.
- Partial one-time-use healing pickups may appear across the dungeon and do not fully heal the player.
- The player carries one unique melee weapon and one unique ranged weapon and can switch between them.
- Character stats and the two weapons have separate permanent upgrade tracks.
- After defeat, the player can spend currency on permanent character and weapon upgrades.
- Previous-self enemies use only a defeated character's stat snapshot and weapon archetypes.
- No movement, input, attack timing, route, pattern, or playstyle recording is used.

- Ordinary previous selves are slightly weaker than the current player; boss selves are stronger and have more HP.
- The first arrival at Floor 5 includes a cutscene revealing the self-boss.
- All untested numerical values are marked as provisional and must be playtested.